Kompilasi Kode Penting & Parameter Kunci (Fase 1 – 3)

Dokumen ini mendokumentasikan implementasi sintaks kode Python/PyTorch/OpenCV/YOLO yang esensial beserta penjelasan mendalam untuk setiap parameter dan fungsi yang telah digunakan hingga Fase 3 (Plate Detector Training).

—

1. Fase 1: Eksplorasi Data, Validasi BBox & Ekstraksi Metadata

Kode pada fase ini berfokus pada parsing aman terhadap file anotasi COCO JSON hierarkis, inspeksi struktur, penanganan anomali tipe data, dan visualisasi bounding box.

1.1 Cuplikan Kode Eksplorasi Struktur COCO JSON & Validasi Geometri

```
import json, cv2, os, glob
import numpy as np
from collections import Counter, defaultdict

# 1. Parsing aman dengan explicit type casting
with open(det_json_path, 'r') as f:
    coco_data = json.load(f)

images_info = {img['id']: img for img in coco_data['images']}
annotations = coco_data['annotations']

# 2. Validasi batas koordinat bounding box (Out-of-bounds check)
invalid_boxes = []
for ann in annotations:
    W = float(images_info[ann['image_id']]['width'])
    H = float(images_info[ann['image_id']]['height'])
    x, y, w, h = [float(v) for v in ann['bbox']]

    if w ≤ 0 or h ≤ 0 or x < 0 or y < 0 or (x + w) > W or (y + h) > H:
        invalid_boxes.append(ann['id'])

# 3. Visualisasi sampel acak dengan bounding box
np.random.seed(42)
random_indices = np.random.choice(len(annotations), size=4, replace=False)
sample_anns = [annotations[i] for i in random_indices]

for ann in sample_anns:
    img_info = images_info[ann['image_id']]
    img_bgr = cv2.imread(os.path.join(det_img_dir, img_info['file_name']))
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    x, y, w, h = [int(float(v)) for v in ann['bbox']]
    cv2.rectangle(img_rgb, (x, y), (x + w, y + h), (0, 255, 0), thickness=3)
```

Parameter	Nilai	Fungsi & Alasan Rekayasa
`float(v)` casting	`Explicit`	Menghindari `TypeError: '>' not supported between 'int' and 'str'` akibat atribut COCO JSON yang tersimpan sebagai string numerik (misal `"width": "1280"`).
`(x+w) > W`	Batas geometris	Memastikan kotak tidak keluar kanvas citra yang berpotensi menghasilkan nilai koordinat negatif atau loss NaN saat normalisasi.
`thickness=3`	`3 px`	Memberikan ketebalan visualisasi yang cukup kontras saat di-render pada citra beresolusi tinggi (1280×720).
`random.seed(42)`	`42`	Menjamin reproduktibilitas; indeks acak yang dipilih selalu sama pada setiap kali notebook dijalankan ulang.

—

2. Fase 2: Preprocessing, Group-Based Splitting & Konversi YOLO

Kode pada fase ini menangani transformasi koordinat absolut ke standar normalisasi Ultralytics YOLO serta memisahkan dataset berdasarkan klaster mobil unik untuk mencegah kebocoran data (**data leakage**).

2.1 Group Splitting Bebas Leakage & Konversi Format BBox

```python
# 1. Ekstraksi identitas dasar mobil (menghapus variasi hash augmentasi)
def get_image_group_key(filename):
    base = os.path.basename(filename)
    if ".rf." in base:
        return base.split(".rf.")[0].replace("_jpg", "").replace("_png", "")
    return os.path.splitext(base)[0]

group_to_ids = defaultdict(list)
for img_id, img_info in images_info.items():
    grp = get_image_group_key(img_info['file_name'])
    group_to_ids[grp].append(img_id)

# 2. Pengacakan pada tingkat grup (mobil unik), bukan baris file
unique_groups = list(group_to_ids.keys())
np.random.seed(42)
np.random.shuffle(unique_groups)

n_groups = len(unique_groups)
n_train_g = int(0.80 * n_groups)
n_val_g = int(0.10 * n_groups)

train_groups = set(unique_groups[:n_train_g])
val_groups = set(unique_groups[n_train_g:n_train_g + n_val_g])
test_groups = set(unique_groups[n_train_g + n_val_g:])

# 3. Normalisasi COCO [x_min, y_min, w, h] → YOLO [class_id, x_c, y_c, w_n, h_n]
def convert_bbox_to_yolo(bbox, img_w, img_h):
    x, y, w, h = [float(v) for v in bbox]
    xc = (x + w / 2.0) / img_w
    yc = (y + h / 2.0) / img_h
    wn = w / img_w
    hn = h / img_h
    # Batasi rentang mutlak 0.0 - 1.0
    xc, yc = max(0.0, min(1.0, xc)), max(0.0, min(1.0, yc))
    wn, hn = max(0.0, min(1.0, wn)), max(0.0, min(1.0, hn))
    return f"0 {xc:.6f} {yc:.6f} {wn:.6f} {hn:.6f}"
```

Parameter	Nilai	Fungsi & Alasan Rekayasa
`split(".rf.")[0]`	String match	Memotong hash Roboflow agar variasi gambar mobil yang sama (hasil rotasi/eksposur) dikelompokkan ke satu kunci yang identik.
`80 / 10 / 10`	Rasio grup	Membagi 633 klaster mobil unik menjadi: Train (506 grup), Val (63 grup), dan Test (64 grup). Menjamin independensi mutlak data evaluasi.
`class_id = 0`	`0`	Label kelas tunggal (`license_plate`) untuk meminimalkan beban komputasi klasifikasi multi-kelas.
`xc = x + w/2`	Titik tengah	Format wajib YOLO yang mengukur titik tengah kotak alih-alih sudut kiri atas seperti COCO/Pascal VOC.

—

3. Fase 3: Training & Evaluasi Model Deteksi (YOLOv8n)

Kode pada fase ini mengonfigurasi mesin pelatihan Ultralytics YOLOv8n dengan GPU CUDA, augmentasi khusus teks, dan evaluasi resmi pada test set independen.

3.1 Konfigurasi Pelatihan Model & Evaluasi Test Set

```python
from ultralytics import YOLO

# Inisialisasi model pre-trained nano COCO (~3.2M parameter)
model = YOLO("yolov8n.pt")

# Training loop dengan parameter yang dioptimalkan untuk plat nomor
results = model.train(
    data="yolo_detector_data.yaml",
    epochs=50,
```

```
    imgsz=640,
    batch=16,
    device=0,           # GPU T4 / CUDA
    workers=4,
    optimizer="auto",  # Otomatis memilih SGD/AdamW berdasarkan gradient stats
    fliplr=0.0,         # SANGAT KRUSIAL: Matikan horizontal flip
    mosaic=1.0,         # Gabungkan 4 citra untuk melatih objek kecil
    degrees=5.0,        # Toleransi kemiringan sudut kamera HP
    scale=0.5,          # Variasi jarak dekat-jauh mobil
    hsv_v=0.4,          # Variasi intensitas pencahayaan (Value)
    hsv_s=0.4,          # Variasi saturasi warna
    save=True,
    project="/kaggle/working/runs_detector",
    name="yolov8n_indonesian_plate"
)

# Evaluasi resmi pada test set independen
best_model = YOLO("/kaggle/working/best_detector.pt")
test_metrics = best_model.val(data="yolo_detector_data.yaml", split="test", imgsz=640)
```

Parameter	Nilai	Fungsi & Alasan Rekayasa
`yolov8n.pt`	`Nano`	Memilih varian teringan (6.2 MB FP32, 3.1 MB FP16). Memenuhi batas kuota deployment mobile ($\leq 6\text{MB}$).
`imgsz=640`	`640 px`	Resolusi input seimbang. Plat nomor hanya berukuran 2% luas citra; resolusi di bawah 640 membuat fitur huruf/angka plat kehilangan detail spasial.
`batch=16`	`16`	Ukuran batch optimal untuk GPU VRAM 16 GB (NVIDIA T4) yang menjaga kestabilan statistik batch normalization.
`fliplr=0.0`	`0.0 (OFF)`	**Paling Krusial**: Membalik gambar secara horizontal membuat karakter plat nomor terbalik seperti cermin, yang merusak ekstraksi fitur orientasi karakter.
`mosaic=1.0`	`1.0 (ON)`	Mengompilasi 4 gambar menjadi 1 frame. Memaksa model mendeteksi objek berskala kecil pada latar belakang yang padat dan heterogen.
`degrees=5.0`	`5.0 derajat`	Mensimulasikan deviasi tangan pengguna saat mengambil foto kendaraan menggunakan smartphone.
`hsv_v=0.4`	`0.4`	Melatih ketahanan fitur terhadap variasi eksposur ekstrim: bayangan pohon, terik matahari siang, maupun kondisi malam hari.

—

4. Rangkuman Metrik Evaluasi Hasil Fase 3

Hasil inferensi resmi bobot terbaik (`best_detector.pt`) pada test set independen:

Metrik Evaluasi	Nilai Capaian	Interpretasi Rekayasa Sistem
Precision @ 0.50	`85.02%`	Dari seluruh kotak yang diprediksi detektor, 85.02% terbukti plat nomor asli (hanya 15% false positive pada grille/rambu).
Recall @ 0.50	**`86.47%`**	**Metrik Paling Kritis**: Detektor berhasil menangkap 86.47% plat nomor yang ada di test set. Nilai ini menjadi batas atas suplai citra ke modul OCR.
mAP @ 0.50	`86.88%`	Rata-rata presisi pada ambang batas IoU 0.50 yang membuktikan konvergensi deteksi objek tunggal sudah sangat baik.
mAP @ 0.50:0.95	`52.86%`	Mengukur presisi ketat tumpang tindih koordinat kotak terhadap ground-truth pada rentang IoU 0.50 hingga 0.95.